

SimDash: Unity Arduino Car Simulator

HTI.320 Group 7 Final Presentation

Donya Davoodi, Jere Kivikangas, Elias Peltonen,
Faezeh Sayad Sijani, Prajjwol Tripathi

December 4, 2024

Tampere University

Introduction

What is SimDash?

- Interactive driving simulator combining hardware and software
- Real-time feedback through physical components
- Unity-based 3D environment with realistic physics
- Arduino-controlled peripherals for immersive experience

Core Concept: Bridge the gap between virtual simulation and tangible interaction

Why Driving Simulation?

Applications:

- Driver training and education
- Behavioral analysis in psychology
- Medical research (stress response)
- Autonomous vehicle development
- Gaming and entertainment

Our Focus:

- Realistic environmental simulation
- Tangible feedback systems
- Foundation for advanced research
- Accessible prototyping approach

Complete Interaction System

Software (Unity):

- 3D driving environment
- Realistic physics engine
- Weather simulation (rain)
- Random event scenarios
- Serial communication interface
- Speed monitoring & warnings

Hardware (Arduino):

- Arduino Uno microcontroller
- 3-digit 7-segment display (5641AS)
- 5V fan motor with PWM control
- Steering wheel input device
- USB serial connection

Bidirectional data flow: Unity ↔ Serial ↔ Arduino

Hardware Components

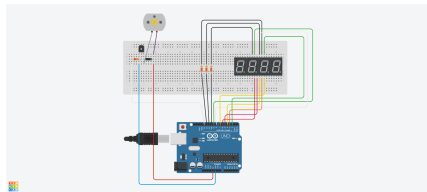
7-Segment Display (5641AS):

- Displays speed (0-100)
- 3-digit common cathode configuration
- Segment pins: 9, 2, 3, 5, 6, 8, 7, 4
- Digit control: pins 10, 11, 12
- Uses SevSeg library for multiplexing

Fan Motor:

- 5V DC motor
- PWM control via pin 13
- Speed mapped from game (0-100 $\rightarrow$ 0-255 PWM)
- Simulates wind flow during driving

Circuit Diagram



Software Features

Core Functionality:

- Steering wheel control integration
- Realistic car physics (friction, collision)
- Dynamic camera positioning
- Real-time speed calculation
- High-speed warning system

Environmental Features:

- Weather effects (rain simulation)
- Audio feedback (engine, rain sounds)
- Urban environment (POLYGON city pack)
- Random scenarios (e.g., animals crossing)
- Day/night lighting

Technology Stack: Unity 6.2, C#, Arduino IDE, SevSeg library

Prototype Demo

Key Demo Features

1. Real-time Speed Display

- Speed from Unity transmitted via serial (9600 baud)
- Arduino parses and displays on 7-segment display
- Updates dynamically as car accelerates/decelerates

2. Wind Flow Simulation

- Fan speed synchronized with car speed
- Provides tactile feedback to driver
- Enhances immersion in virtual environment

3. Interactive Driving Environment

- Steering wheel controls car direction
- Physics-based movement and collision
- Weather effects impact visual experience

Why these features?

- **7-Segment Display:** Simple, reliable, real-world instrument panel feel
- **Fan Motor:** Cost-effective haptic feedback, easy to implement
- **Unity Physics:** Realistic simulation without complex custom code
- **Weather Effects:** Demonstrates environmental variable handling
- **Serial Communication:** Industry-standard protocol, well-documented

Core Concept Illustration:

Virtual actions produce **tangible, real-world responses**

Creating a **closed feedback loop** between digital and physical realms

Proposal vs. Final Product

Initial Proposal Included:

- Heart rate sensor for stress monitoring
- ECG helmet for brain signal analysis
- Automatic speed control based on driver stress
- Behavioral analysis during driving
- Medical/psychological research applications

Ambitious scope for rapid prototyping timeline

What Changed?

Removed Features:

- Heart rate sensor
 - Delayed delivery
 - Time constraints
- ECG helmet integration
 - Complex implementation
 - Limited availability
 - Scope too large
- Automatic stress-based control

Enhanced Instead:

- Polished driving environment
- Improved weather effects
- Better audio integration
- More stable hardware setup
- Refined physics simulation
- Random scenario system

Result: More focused, achievable prototype that still demonstrates core concepts

Final Deliverable Alignment

Feature	Proposed	Delivered
Steering wheel control	✓	✓
Speed display (7-segment)	✓	✓
Fan motor (wind simulation)	✓	✓
Unity 3D environment	✓	✓
Weather simulation	✓	✓
Physics engine	✓	✓
Random scenarios	✓	✓
Audio feedback	✓	✓
Heart rate sensor	✓	×
ECG integration	✓	×

Completion Rate: 80% of proposed original features, with quality improvements

Development Process

Software Development (Unity)

1. Environment Design

- 3D scene creation with POLYGON city pack
- Camera positioning and lighting setup

2. Physics Implementation

- Rigidbody dynamics for car movement
- Acceleration and collision detection
- Simple steering wheel input mapping

3. Serial Communication

- Data formatting and transmission protocols
- Error handling and connection management

4. Feature Integration

- Weather systems, audio, warnings
- Testing and synchronization with hardware

Hardware Development (Arduino)

1. Component Testing

- Individual testing of display and motor
- Pin configuration validation
- Power requirements verification

2. Display Integration

- SevSeg library configuration
- 3-digit multiplexing setup
- Brightness and refresh rate optimization

3. Motor Control

- PWM signal generation
- Speed mapping algorithm (0-100 $\rightarrow$ 0-255)
- Transistor circuit for current handling

4. Serial Protocol

- Data parsing from Unity
- Standalone test mode development

Challenges & Solutions:

- **Power Management**

- Challenge: Motor current draw
- Solution: Transistor-based switching circuit

- **Display Limitations**

- Challenge: 4-digit display, but pin 13 needed for motor
- Solution: Use only 3 digits (sufficient for speed 0-100)

Tools & Workflow:

▪ GitHub Repository

- Version control for code
- Branch-based development
- Main branch stability
- Merge reviews

▪ Miro Board

- Visual planning
- Task tracking
- Milestone setting

▪ Documentation

- README.md (setup guide)
- Circuit diagrams
- Code comments
- Final report

▪ Testing Approach

- Standalone Arduino tests
- Unity scene testing
- Integrated system validation

Repository: <https://github.com/3liasP/simdash>

Deliverables & Outcomes

Three Deliverables Overview

1. Initial Proposal & Planning

- Set ambitious vision with heart rate/ECG
- Established clear milestones
- Created Miro board for collaboration

2. Midterm Progress Report

- Hardware sourcing challenges identified
- Scope refinement decisions made
- Core functionality demonstrated

3. Final Prototype & Report

- Working hardware-software integration
- Polished driving experience
- Complete documentation

Deliverable Outcomes & Improvements

What Went Well:

- Early milestone setting provided direction
- GitHub workflow enabled parallel development
- Flexibility to adjust scope when needed
- Final product matches refined vision

Possible Improvements:

- **Task Division:** More explicit role assignments from the start
- **Communication:** Regular sync meetings beyond class time
- **Hardware Planning:** Earlier component ordering and verification
- **Scheduling:** Better coordination for in-person hardware work

Lessons Learned

Key Technical Learnings:

- **Unity Basics:** Several members learned Unity from scratch
- **GitHub Proficiency:** Version control skills developed across team
- **Serial Communication:** Understanding of hardware-software protocols
- **Arduino Programming:** Microcontroller development experience
- **Circuit Design:** Practical electronics knowledge gained

Individual Reflections:

- Initial learning curve was steep for less experienced members
- Step-by-step approach helped build confidence
- Cross-learning between team members valuable
- Hands-on experience more effective than theoretical study

Team Insights: Process

What Worked:

- Freedom to choose tasks based on interest/skill
- Early tool adoption (GitHub, Miro)
- Willingness to help each other learn
- Flexibility in adjusting scope

Challenges Faced:

- **Unclear task division** led to some confusion
- **Limited prior experience** with technologies as a team
- **Busy schedules** restricted meeting opportunities
- **Merge conflicts** from uncoordinated parallel work

Rapid Prototyping Insights

Key Lessons:

- **Scope Management:** Start smaller, iterate larger
- **Hardware Risks:** Component availability is important from the start
- **Agile Balance:** "Move fast" doesn't mean "skip docs/planning"
- **Integration Early:** Test hardware-software connection ASAP
- **Parallel Work:** Requires excellent coordination to avoid conflicts
- **Flexibility:** Be ready to pivot when obstacles arise

**Despite challenges, we delivered a functional prototype
that demonstrates our core concept successfully!**

Thank You!

Questions?

Repository:



Miro Board:

